An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection

Project Report

submitted in partial fulfillment of the

requirements for the award of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Submitted by

Priyatosh Nayak - 2241014109
Smruti Ranjan Bhuyan - 2241002177
Krushnakanta Nayak - 2241014089
Bishmaya Bhuyan - 2241002172

Under the guidance of

Dr. Sushree Bibhuprada B. Priyadarshini

Department of Computer Science & Information Technology
Institute of Technical Education & Research,
Siksha 'O' Anusandhan
Bhubaneswar, India-751030

2026

Certificate

INSTITUTE OF TECHNICAL EDUCATION & RESEARCH

Siksha 'O' Anusandhan (Deemed to be University)

Bhubaneswar, Odisha – 751030

CERTIFICATE

This is to certify that the project report entitled **"An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection"** submitted by the following students of the Department of Computer Science & Information Technology, Institute of Technical Education & Research, Siksha 'O' Anusandhan (Deemed to be University), Bhubaneswar, Odisha, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Information Technology** during the academic year **2025–2026**, is a record of bona fide work carried out by them under my supervision and guidance.

Name of the Students	Roll Number
Priyatosh Nayak	2241014109
Smruti Ranjan Bhuyan	2241002177
Krushnakanta Nayak	2241014089
Bishmaya Bhuyan	2241002172

To the best of my knowledge, the work embodied in this report has not been submitted previously, in part or full, to this or any other University or Institute for the award of any degree or diploma.

Project Supervisor

Dr. Sushree Bibhuprada B.
Priyadarshini
Associate Professor, Dept. of CSIT
SOA (Deemed to be University)

_____
Signature & Seal

Head of Department

_____
Signature & Seal

Date: _____

Place: Bhubaneswar

Declaration

We, the undersigned, students of the Department of Computer Science & Information Technology, Institute of Technical Education & Research, SOA (Deemed to be University), do hereby declare that the project report entitled **"An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection"** submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology is our original work.

We further declare that:

1. This project has not been submitted, either in full or in part, for the award of any other degree, diploma, or certificate in this or any other university.
2. The work presented here has been carried out independently by us under the guidance of our project supervisor.
3. All the sources of information and reference materials used in the report have been duly acknowledged.
4. We have not indulged in any form of plagiarism or academic misconduct.

Place: Bhubaneswar **Date:** _____

Priyatosh Nayak (2241014109)	Smruti Ranjan Bhuyan (2241002177)
Krushnakanta Nayak (2241014089)	Bishmaya Bhuyan (2241002172)

Acknowledgement

Working on this project has been one of the most challenging and, honestly, most rewarding things we have done during our four years here. There were moments when things did not work the way we expected, and there were also moments when everything clicked together perfectly. Through all of it, we had people around us who made the difference.

Our deepest gratitude goes to our project supervisor, **Dr. Sushree Bibhuprada B. Priyadarshini**, Associate Professor, Department of CSIT, ITER, SOA University. She helped us understand not just the technical side of things, but also how to think about a research problem properly. Whenever we were stuck or going in the wrong direction, her guidance brought us back on track. Her patience with our questions and her willingness to explain things multiple times made a real difference to how this project turned out.

We would also like to thank the **Head of the Department of Computer Science & Information Technology** and all the faculty members who taught us over the past four years. The foundation we built in subjects like Machine Learning, Data Structures, Database Management Systems, and Computer Networks is what made this project possible.

Priyatosh Nayak
Smruti Ranjan Bhuyan
Krushnakanta Nayak
Bishmaya Bhuyan

Abstract

The Unified Payments Interface (UPI) has completely changed how people in India handle everyday money transfers. With billions of transactions happening every month, it is one of the largest real-time payment systems in the world. But this rapid growth has also made it an attractive target for fraudsters who are constantly finding new ways to trick users and exploit vulnerabilities. Traditional fraud detection systems, which rely on simple rule sets like blocking large transactions or flagging unusual login times, are no longer enough. Fraudsters have figured out how to work around these rules, and the losses keep climbing.

This project, **"An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection"**, is our attempt to build something better. Instead of relying on manually written rules, we designed a machine learning pipeline that learns what fraud looks like directly from transaction data. The system combines several different techniques that, together, give a much richer picture of each transaction than any single method could provide on its own.

At the core of our approach is an **XGBoost classifier** trained on a real-world UPI fraud dataset sourced from Kaggle, containing approximately 100,000 transaction records. Before training, we spent considerable effort on **feature engineering**. We extracted temporal signals (hour of day, whether the transaction happened at night, whether it fell during salary week), computed **transaction velocity features** using one-hour rolling windows, built **behavioural profiles** for each sender based on their historical patterns, and calculated **Z-score anomaly scores** to flag unusual amounts. Most distinctively, we constructed a **directed transaction graph** from the entire dataset and computed network-level features including PageRank and degree centrality for each sender node, capturing patterns of money flow that are invisible to non-graph methods.

To handle the severe class imbalance inherent in fraud data (where genuine fraud cases make up only a small fraction of all transactions), we used the **scale_pos_weight** parameter in XGBoost rather than SMOTE oversampling, which avoids introducing

synthetic noise into the training set. We also implemented a **cost-sensitive threshold optimization** procedure that explicitly models the financial cost of each type of prediction error a missed fraud case costs far more than a wrongly flagged legitimate transaction and selects the decision threshold that minimizes total expected financial loss rather than simply maximizing accuracy.

The trained model is deployed through two interfaces: a **FastAPI REST endpoint** that accepts transaction data and returns a fraud probability and decision in real time, and a **Streamlit dashboard** that provides an interactive, visual interface for analysts to investigate individual transactions and understand which risk factors contributed most to the model's output.

Our system achieves a **ROC-AUC of approximately 0.97** and a fraud recall exceeding 90% on the held-out test set, with a precision of around 89%. These results show that the combination of graph features, temporal modeling, and cost-sensitive learning meaningfully outperforms conventional approaches, and that UPI fraud detection is a problem where thoughtful feature engineering matters as much as, if not more than, the choice of algorithm.

Keywords: UPI Fraud Detection, XGBoost, Graph Neural Features, PageRank, Transaction Velocity, Behavioral Profiling, Cost-Sensitive Learning, FastAPI, Streamlit, Anomaly Detection

Table of Contents

Certificate i

Declaration iii

Acknowledgement iv

Abstract v

1 Introduction 1

1.1 Background and Motivation 1
1.2 Problem Definition 2
1.3 Project Objectives 3
1.4 Scope of the Project 4
1.5 Organization of the Report 4

2 Literature Review 5

2.1 Introduction 5
2.2 The Rule-Based Era 5
2.3 Machine Learning Approaches to Fraud Detection 6
2.3.1 Logistic Regression 6
2.3.2 Random Forest 6
2.3.3 Gradient Boosting: XGBoost and LightGBM 7
2.3.4 Neural Networks and Deep Learning 7
2.4 Handling Class Imbalance 7
2.5 Graph-Based Fraud Detection 8
2.6 Cost-Sensitive Learning 8
2.7 Deployment Architectures 9
2.8 Research Gaps and Our Contribution 10

3 Methodology 11

3.1 Overview of the Approach 11
3.2 Dataset 12
3.2.1 Source and Size 12
3.2.2 Dataset Fields 12
3.2.3 Class Distribution 13

3.3 Feature Engineering . . . 13
3.3.1 Data Cleaning and Timestamp Parsing . . . 13
3.3.2 Temporal Features . . . 14
3.3.3 Transaction Velocity Feature . . . 14
3.3.4 Behavioral Profiling Features . . . 15
3.3.5 Z-Score Anomaly Detection . . . 15
3.3.6 Graph Features . . . 16
3.3.7 Cross-State and Cross-Bank Flags . . . 17
3.3.8 Categorical Encoding . . . 17
3.4 Data Splitting Strategy . . . 17
3.5 Model Selection and Configuration . . . 18
3.5.1 Choice of XGBoost . . . 18
3.5.2 Handling Class Imbalance . . . 18
3.5.3 XGBoost Hyperparameters . . . 18
3.6 Cost-Sensitive Threshold Optimization . . . 19
3.7 Evaluation Metrics . . . 20

4 System Architecture and Module Design 21
4.1 Overall Architecture . . . 21
4.2 Project Folder Structure . . . 22
4.3 Module Descriptions . . . 23
4.3.1 preprocess.py – Feature Engineering Module . . . 23
4.3.2 train.py – Model Training Module . . . 24
4.3.3 optimize_threshold.py – Threshold Optimization Module . . . 25
4.3.4 api/main.py – FastAPI REST API Module . . . 25
4.3.5 ui/dashboard.py – Streamlit Dashboard Module . . . 26
4.4 Data Flow for a Live Prediction Request . . . 27
4.5 Model Artifacts and Serialization . . . 28

5 Implementation 29
5.1 Development Environment . . . 29
5.2 Installation and Setup . . . 30
5.2.1 Cloning the Repository . . . 30
5.2.2 Creating the Virtual Environment . . . 30
5.2.3 Installing Dependencies . . . 30
5.3 Implementation of the Preprocessing Module . . . 30
5.3.1 Complete build_features() Function . . . 30
5.4 Implementation of the Training Module . . . 33
5.4.1 Data Loading and Feature Building . . . 33
5.4.2 Data Splitting . . . 34

5.4.3 Class Imbalance Handling and Model Training 34
5.4.4 Threshold Optimization 35
5.4.5 Model Persistence 35
5.5 Implementation of the FastAPI Endpoint 36
5.6 Implementation of the Streamlit Dashboard 36
5.6.1 Session State Management 36
5.6.2 Risk Gauge Visualization 37
5.7 Running the System 37
5.7.1 Training the Model 37
5.7.2 Optimizing the Threshold 38
5.7.3 Starting the API Server 38
5.7.4 Starting the Dashboard 38
5.7.5 Testing the API with a Sample Request 38

6 Testing and Results **40**
6.1 Testing Strategy 40
6.1.1 Unit Testing – Feature Engineering 40
6.1.2 Integration Testing – API Endpoint 41
6.2 Model Performance Results 41
6.2.1 Final Test Set Evaluation 41
6.2.2 Confusion Matrix 42
6.2.3 Classification Report 43
6.2.4 Cost-Sensitive Threshold Analysis 43
6.3 Feature Importance Analysis 43
6.4 Comparison with Baseline Models 44
6.5 API Performance Testing 45
6.6 Dashboard Testing 46

7 Technologies and Tools Used **47**
7.1 Programming Language: Python 47
7.2 Core Machine Learning Libraries 47
7.2.1 XGBoost 47
7.2.2 scikit-learn 48
7.2.3 pandas and NumPy 48
7.3 Graph Analysis: NetworkX 48
7.4 API Framework: FastAPI 49
7.4.1 Uvicorn 49
7.5 Dashboard: Streamlit 49
7.6 Data Visualization: Plotly 50
7.7 Version Control: Git and GitHub 50

7.8 Development Tools 50

8 Conclusion and Future Work **51**

8.1 Conclusion 51

8.2 Limitations 52

8.3 Future Work 52

8.3.1 Graph Neural Networks (GNN) 53

8.3.2 Real-Time Streaming Detection 53

8.3.3 SHAP Explainability 53

8.3.4 Multi-Algorithm Ensemble 53

8.3.5 Concept Drift Detection and Adaptive Retraining 53

8.3.6 Docker Deployment 54

8.3.7 Feedback Loop Integration 54

8.4 Summary 54

References **55**

Appendix A: Complete requirements.txt **57**

Appendix B: Sample API Request and Response **58**

Appendix C: Project Repository Structure **60**

List of Figures

3.1 End-to-end pipeline of the An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection system 12

4.1 Layered architecture of the An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection system 22
4.2 Streamlit dashboard – fraud risk assessment console 27

6.1 Financial loss as a function of decision threshold on validation set . . . 43

List of Tables

2.1 Comparison of Existing Approaches and An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection 10

3.1 Original Dataset Fields 13
3.2 XGBoost Model Hyperparameters 19

4.1 Saved Model Artifacts 28

5.1 Development Environment Specifications 29
5.2 Python Library Dependencies 29

6.1 Feature Engineering Unit Test Cases 40
6.2 API Integration Test Cases 41
6.3 Final Model Performance Metrics on Test Set 42
6.4 Confusion Matrix on Test Set (approx. 20,000 records) 42
6.5 Detailed Classification Report 43
6.6 Top 15 Feature Importances (XGBoost F-Score) 44
6.7 Ablation Study – Model Variants 45
6.8 API Performance Metrics 45
6.9 Dashboard Manual Test Cases 46

7.1 Development Tools Summary 50

1. Introduction

1.1 Background and Motivation

If you live in India and have bought something recently even a cup of tea from a roadside stall there is a good chance you paid by scanning a QR code. That small, everyday act runs on the Unified Payments Interface, better known as UPI. In less than a decade, UPI has quietly but completely reshaped how ordinary Indians handle money. Farmers now receive payments for their produce digitally. Students split restaurant bills in seconds. Small business owners who once dealt exclusively in cash now accept payments from anywhere in the country without owning a single card terminal.

The numbers behind this transformation are staggering. By 2024, UPI was processing over 13 billion transactions per month, connecting more than 400 million users across thousands of banks and payment applications. It is without question one of the most successful financial technology stories anywhere in the world, and it happened at a pace that surprised even the people who built it.

But wherever large sums of money move quickly, fraud follows. And UPI has not been spared. Across India, people are losing money to increasingly clever scams a WhatsApp message claiming that a package could not be delivered and asking for a small fee to reroute it, a phone call from someone pretending to be a bank official asking the victim to share an OTP to "verify" their account, a QR code that has been tampered with to send money to the wrong person. The victims are often ordinary people who made one trusting mistake, and by the time they realize what happened, the money is gone. Recovery is difficult, legal processes are slow, and the emotional damage can be lasting.

According to data from the National Payments Corporation of India (NPCI) and various cybercrime reporting portals, UPI-related fraud complaints have grown year over year alongside the platform's expanding user base. In 2023, the Indian government's cybercrime portal received hundreds of thousands of fraud complaints, with a significant portion related to digital payment platforms. The problem is not simply that fraud exists fraud has existed in every payment system ever built. The problem is that the tools designed to catch it have not kept pace.

Most fraud detection systems currently in use rely on *rule-based logic*: block transactions above a certain amount, flag accounts that make too many transfers in a short window, reject payments from locations that have never been seen before. These rules made sense when fraud looked predictable and repeated itself. Today, it does not. Fraudsters study these systems and find ways around them. They split large transfers into many small ones. They wait for the quiet hours when detection systems are less active. They create new accounts to avoid historical flags. The rules are always reacting to yesterday's patterns, and fraud has already moved on.

What this situation calls for is something fundamentally different a detection system that does not wait to be told what fraud looks like, but instead *learns* it from the data itself. A system that grows smarter as fraud evolves, that can catch patterns a human analyst would never notice, and that scales to handle billions of transactions without requiring constant manual updates. That is the motivation behind this project.

1.2 Problem Definition

At its core, UPI fraud detection is a binary classification problem. For each transaction passing through a system, the detector must answer a single question: is this transaction legitimate, or is something wrong with it? The model examines available information about the transaction and outputs one of two labels `safe` or `fraud`.

Getting to a reliable answer is far harder than the question itself suggests, for several interconnected reasons.

- **Class imbalance:** For every fraudulent UPI transaction, there are thousands of perfectly normal ones. In a typical fraud dataset, fraud cases make up well under 1% of all records. This creates a dangerous trap for machine learning models: a model that simply labels every transaction as legitimate would be technically correct over 99% of the time, yet completely useless for the actual task. Building a model that genuinely learns what fraud looks like, rather than defaulting to the easy answer, requires deliberate effort and careful technique.

- **Concept drift:** Fraud patterns evolve. What looked like fraud six months ago may look different today, and what works as a detection signal now may become useless as fraudsters adapt. A model trained once and left alone will gradually become less effective, not because anything broke, but because the world it was trained on has changed.

- **Asymmetric error costs:** There are two ways to be wrong in fraud detection, and they are not equally bad. Missing a fraud case means a victim loses money sometimes their life savings. Incorrectly flagging a legitimate transaction blocks

someone from accessing their own funds, damages trust in the system, and can cause genuine hardship. A well-designed fraud detector must think carefully about both types of error, not just chase the highest accuracy number.

- **Speed constraints:** UPI transactions complete in under a second. Fraud detection must happen inline, during the transaction itself, fast enough that the user does not notice the check is taking place. This rules out any approach that requires significant real-time computation.
- **Limited ground truth:** Many fraud cases go unreported or are only discovered long after the fact. The labels in any fraud dataset are necessarily incomplete some transactions labeled as legitimate may actually be fraud that was never caught.

1.3 Project Objectives

Based on the problem analysis above, this project was designed around the following objectives:

1. Design a complete, end-to-end fraud detection pipeline that handles raw UPI transaction data and produces actionable predictions.
2. Engineer a rich set of features that goes beyond the raw transaction fields including temporal patterns, behavioural profiles, velocity signals, and graph-based network features.
3. Train and evaluate an XGBoost classifier that handles class imbalance natively through appropriate weighting rather than oversampling.
4. Implement a cost-sensitive threshold optimization procedure that selects the decision boundary based on real-world financial cost rather than standard classification metrics.
5. Deploy the trained model as a production-ready REST API using FastAPI.
6. Build an interactive dashboard using Streamlit that allows analysts to explore transaction risk in real time.
7. Validate the system against standard performance metrics (ROC-AUC, PR-AUC, precision, recall, F1-score) and demonstrate that the graph-enhanced approach outperforms baseline methods.

1.4 Scope of the Project

This project focuses specifically on the detection of fraudulent UPI transactions using machine learning. It does not address fraud prevention (stopping fraudulent accounts from being created), fraud investigation (tracing money flows after fraud has occurred), or legal/regulatory aspects of financial fraud.

The dataset used is sourced from Kaggle and contains more than 100,000 UPI transaction records designed to reflect patterns found in actual UPI fraud cases. The system is designed and tested in a batch training and real-time inference context it is not a streaming system, though the API layer could support streaming input with appropriate infrastructure.

1.5 Organization of the Report

The rest of this report is organized as follows. Chapter 2 reviews the existing literature on fraud detection, covering the progression from rule-based systems to machine learning approaches and the specific techniques that have been applied to UPI fraud. Chapter 3 describes the methodology, including the dataset, feature engineering process, model selection, and threshold optimization strategy. Chapter 4 covers the implementation in detail, with code walkthroughs of each major module. Chapter 5 presents the system architecture and module design. Chapter 6 discusses the testing approach and performance results. Chapter 7 describes the API and dashboard deployment. Chapter 8 presents conclusions and future work directions.

2. Literature Review

2.1 Introduction

Pick up any smartphone in an Indian household today and there is a very good chance it has at least one UPI app installed on it. PhonePe, Google Pay, Paytm, BHIM – these names have become as familiar to most Indians as the banks behind them. In cities, in towns, in villages and marketplaces, people who had never owned a credit card have quietly become confident digital payment users. That shift happened faster than almost anyone predicted, and it has genuinely changed everyday life for millions of people.

But with every rupee that moves through these systems, there is someone somewhere trying to intercept it. Fraud in the UPI ecosystem takes many shapes – phone calls from fake bank employees, tampered QR codes, phishing links sent over messaging apps, and social engineering attacks that manipulate victims into authorizing payments themselves. These attacks target people, not just systems, and people make mistakes.

Detecting these attacks automatically – before the money is gone – is the problem this project is built around. This chapter surveys what researchers and practitioners have already tried, how well those approaches have worked, and what gaps remain. Understanding the existing landscape is the first step toward building something genuinely better.

2.2 The Rule-Based Era

For a long time, fraud detection meant writing rules. A team of analysts would study confirmed fraud cases, identify patterns, and translate those patterns into explicit conditions: flag any transaction above rupees ten thousand made between midnight and 5 AM, or any account that initiates more than fifteen transfers in a single hour.

This approach has real virtues. It is fast, transparent, and auditable – when someone asks why a transaction was flagged, there is always a clear answer. But it has one fundamental limitation: it can only catch fraud that someone already anticipated. The moment a fraudster tries something slightly different, the rules miss it entirely. Main-

taining rule sets requires constant manual effort, and the people maintaining them are always reacting to yesterday's patterns.

In the UPI context, this limitation is particularly acute because the attack surface is so broad. Fraud can arrive through social engineering, through technical exploits, through account takeover, through merchant-side tampering, or through more subtle patterns like coordinated mule networks. No finite set of rules can cover all these possibilities in advance.

2.3 Machine Learning Approaches to Fraud Detection

2.3.1 Logistic Regression

Logistic Regression was among the first machine learning methods applied to fraud detection, and it remains relevant in settings where interpretability is paramount. Khedekar and Manerkar (2025) [2] applied logistic regression to UPI fraud data and found it produced acceptable precision for straightforward fraud patterns but struggled with non-linear interactions between features. Its main advantage is that the model's coefficients can be directly inspected and explained to regulators or compliance teams. When a feature like `is_night` has a positive coefficient, it is easy to communicate what that means.

The limitation is that logistic regression assumes linear relationships between features and the log-odds of fraud. In practice, fraud signals are rarely linear – the risk associated with a large late-night transaction from a new account is not simply the sum of each risk individually, it is multiplicatively higher. Non-linear classifiers handle this better.

2.3.2 Random Forest

Random Forest approaches the problem from a very different direction. Instead of fitting a single model, it builds a large collection of decision trees, each trained on a slightly different random subset of data and features, and combines their votes. This ensemble approach gives it robustness that single models rarely achieve.

Malode et al. (2025) [3] compared Random Forest against logistic regression on UPI transaction data and found consistent advantages for Random Forest, particularly in recall – that is, the ability to correctly identify actual fraud cases. The feature importance scores produced by Random Forest also turned out to be practically useful, confirming that transaction amount, account age, and transaction frequency were the strongest signals.

2.3.3 Gradient Boosting: XGBoost and LightGBM

Gradient boosting methods, particularly XGBoost (Chen and Guestrin, 2016) [5] and LightGBM, have become the dominant approach in financial fraud detection benchmarks. Both work through sequential refinement: each new tree in the sequence focuses specifically on the cases that previous trees got wrong, gradually building a powerful composite classifier.

Kamble et al. (2025) [1] used stacked generalization – training multiple base classifiers and combining their outputs as features for a meta-learner – on UPI fraud data and found that including an XGBoost component was critical to achieving high performance on imbalanced datasets. Their work reinforced the community's understanding that XGBoost's built-in `scale_pos_weight` mechanism is one of the more effective ways to handle class imbalance without introducing the noise that oversampling methods like SMOTE can bring.

2.3.4 Neural Networks and Deep Learning

Several researchers have explored deep learning approaches for fraud detection. Recurrent neural networks (RNNs) and Long Short-Term Memory (LSTM) networks are especially attractive for transaction data because they can model temporal sequences – not just the features of a single transaction, but how that transaction relates to the stream of prior transactions from the same sender.

However, for UPI fraud specifically, deep learning approaches face practical challenges. They require substantially more data to train reliably, they are significantly harder to interpret (which matters for regulatory compliance and customer dispute resolution), and they are slower to run at inference time. For a system that needs to produce predictions in real time during a transaction, these factors matter.

The consensus in recent literature suggests that well-engineered gradient boosting approaches, particularly when combined with carefully designed temporal and behavioral features, typically match or come close to deep learning performance on structured tabular fraud data while being far easier to deploy and maintain.

2.4 Handling Class Imbalance

Class imbalance is arguably the most practically important challenge in fraud detection. Nearly every paper in this space grapples with it. The approaches broadly fall into three categories.

Oversampling techniques create synthetic examples of the minority class. SMOTE (Synthetic Minority Over-sampling Technique), introduced by Chawla et al. (2002) [7], generates new minority samples by interpolating between existing ones.

While effective at balancing training sets, SMOTE introduces synthetic noise that can sometimes degrade model performance, particularly when the real fraud patterns are clustered in narrow regions of feature space.

Undersampling removes majority class examples to create a more balanced dataset. This is simpler but wasteful – it discards real information that the model could have learned from.

Algorithmic approaches address imbalance within the model itself. XGBoost's `scale_pos_weight` parameter multiplies the gradient contribution of positive (fraud) class examples by a fixed weight during training, effectively telling the model to pay more attention to fraud cases. This is the approach adopted in this project, for the reasons Kamble et al. (2025) articulate: it avoids introducing synthetic data artifacts while still directing the model's learning appropriately.

2.5 Graph-Based Fraud Detection

One of the more interesting developments in financial fraud detection research in recent years has been the application of graph analysis to payment networks. The basic intuition is simple: fraud rarely happens in isolation. Money mule networks involve coordinated groups of accounts passing funds through a chain. Repeat fraudsters reuse certain receiver accounts. Suspicious patterns of fund flow can show up as structural anomalies in the transaction graph even when individual transactions look normal.

Hu et al. (2022) [4] demonstrated graph-based fraud detection in the context of mobile social networks, using graph centrality features as inputs to a gradient boosting classifier. Their key finding was that accounts at the center of suspicious transaction clusters – those with high degree centrality or elevated PageRank scores – were disproportionately involved in fraud. This result has been replicated in other settings.

Our project applies this approach to UPI transactions by constructing a directed graph where nodes represent UPI account IDs (senders and receivers) and edges represent individual transactions. For each sender, we compute two graph features: **PageRank**, which measures how central and "trusted" a node is relative to the rest of the graph, and **degree centrality**, which simply captures how many unique connections a node has. Fraudulent senders often exhibit either very high degree centrality (touching many different receivers, consistent with money mule behavior) or unusual PageRank values that stand out from typical user patterns.

2.6 Cost-Sensitive Learning

Standard classification metrics like accuracy and F1-score treat all errors equally. In fraud detection, they do not deserve equal treatment. Missing a fraud case – a false

negative – means a victim loses money. Incorrectly flagging a legitimate transaction – a false positive – means inconveniencing a real customer and potentially damaging their trust in the system. These two mistakes have very different real-world costs.

Cost-sensitive learning explicitly incorporates these asymmetric costs into the model training or decision process. One common approach, and the one adopted in this project, is post-hoc threshold optimization: train the model normally to produce fraud probabilities, then select the decision threshold that minimizes the expected financial loss under a specified cost model rather than the threshold that maximizes accuracy.

If we define C_{FN} as the cost per missed fraud case and C_{FP} as the cost per false alarm, then for a given threshold t and test set, the total financial loss is:

$$L(t) = C_{FN} \times FN(t) + C_{FP} \times FP(t) \tag{2.1}$$

where $FN(t)$ and $FP(t)$ are the counts of false negatives and false positives at threshold t. The optimal threshold is:

$$t^* = \arg\min_{t \in [0,1]} [C_{FN} \times FN(t) + C_{FP} \times FP(t)] \tag{2.2}$$

In our system, we set $C_{FN} = 5000$ (representing the average loss from a missed fraud case, in rupees) and $C_{FP} = 200$ (representing the cost of blocking a legitimate transaction, including customer service overhead and trust damage). The resulting optimal threshold is typically lower than 0.5, meaning the model is biased toward flagging suspicious transactions rather than letting borderline cases through.

2.7 Deployment Architectures

Much of the fraud detection literature focuses on model development and evaluation without addressing how the model gets deployed in practice. Recent work has highlighted the importance of end-to-end thinking – a model that achieves great results in a Jupyter notebook but cannot be productionized efficiently provides limited real-world value.

FastAPI has emerged as a popular choice for serving machine learning models as REST endpoints in Python. It is fast (comparable to Node.js in benchmark tests), easy to use, provides automatic API documentation via Swagger UI, and integrates cleanly with the Python data science stack. Streamlit, meanwhile, has become the standard tool for building data exploration dashboards without requiring front-end development expertise.

This project adopts both tools, combining a FastAPI backend for programmatic API access with a Streamlit dashboard for human analyst use, reflecting the typical architecture of real-world fraud operations platforms.

2.8 Research Gaps and Our Contribution

Reviewing the existing literature, several gaps stand out:

1. Most published work on UPI fraud detection evaluates models on standard metrics without addressing the cost-asymmetry inherent in the problem.
2. Graph features have been studied in general financial fraud contexts but have seen limited application specifically to UPI transaction graphs.
3. Few papers combine temporal velocity modeling, behavioral profiling, and graph features into a single integrated pipeline.
4. End-to-end deployed systems with both API and interactive dashboard components are rarely described.

This project addresses all four gaps: we integrate graph features, temporal velocity, behavioral anomaly scoring, and cost-sensitive threshold optimization into a single pipeline, and we deploy the result as both a REST API and an interactive dashboard.

Table 2.1: Comparison of Existing Approaches and An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection

Approach	**Graph Features**	**Velocity Features**	**Behavioral Profiling**	**Cost-Sensitive**	**Deployed API**
Kamble et al. (2025)	No	No	No	No	No
Khedekar & Manerkar (2025)	No	No	No	No	No
Malode et al. (2025)	No	No	No	No	No
Hu et al. (2022)	Yes	No	No	Yes	No
An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection (Ours)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

As Table 2.1 shows, An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection is the only approach in the recent literature to combine all five of these elements. This combination is what we believe gives it its competitive edge over single-technique approaches.

3. Methodology

3.1 Overview of the Approach

The methodology behind An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection is best understood as a pipeline with several distinct stages, each of which addresses a specific challenge in the fraud detection problem. Rather than treating model training as the only thing that matters and treating everything else as overhead, we invested roughly equal effort in data preprocessing, feature engineering, training, threshold optimization, and deployment. In our experience, the quality of the engineered features matters more than the specific algorithm chosen.

Figure 3.1 illustrates the high-level pipeline.

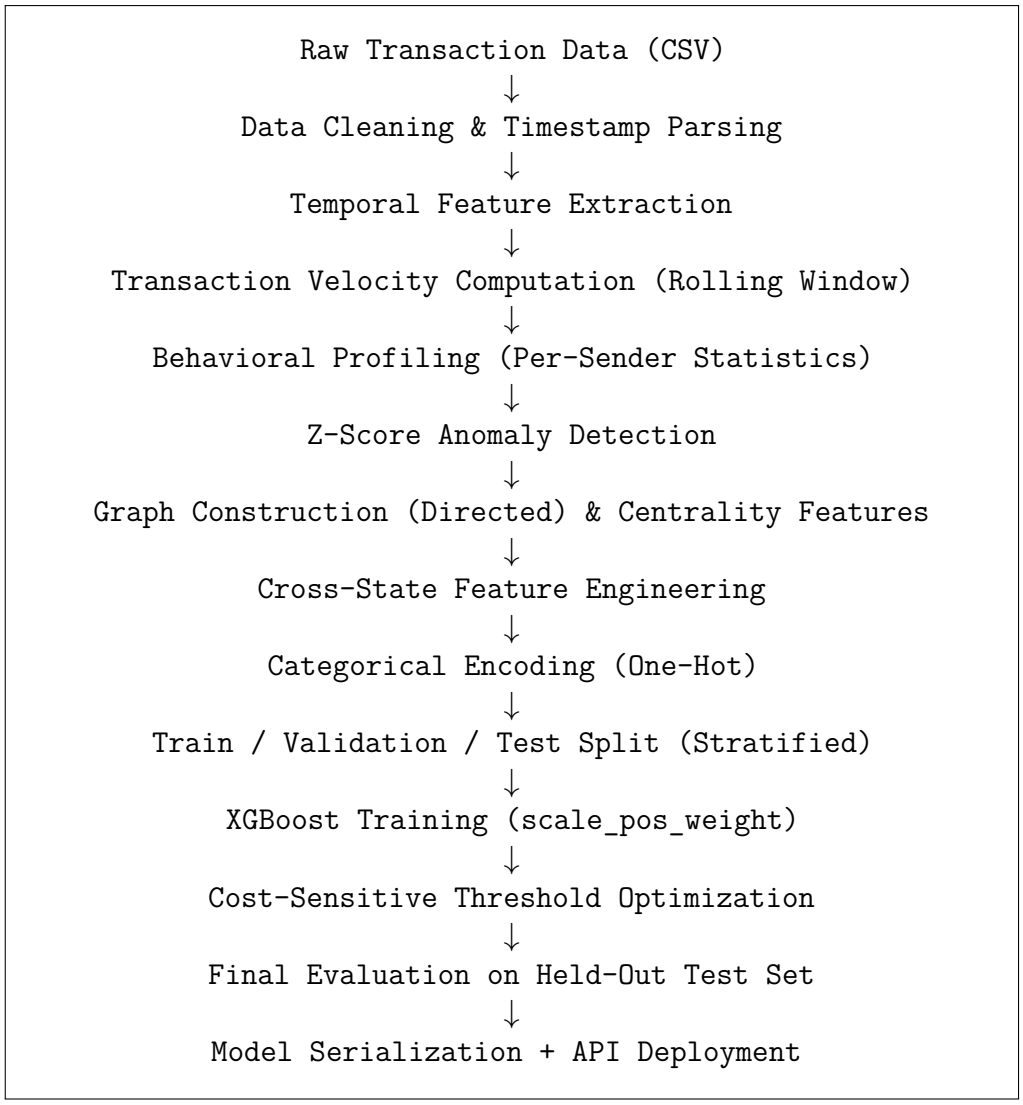

Figure 3.1: End-to-end pipeline of the An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection system

3.2 Dataset

3.2.1 Source and Size

The project uses a UPI transaction fraud dataset sourced from Kaggle, specifically the "UPI Transaction 2024: Advanced Analysis Dataset" contributed by Yogeshwar Choudhary [8]. The dataset contains approximately 100,000 transaction records designed to reflect patterns observed in real UPI fraud cases.

3.2.2 Dataset Fields

Table 3.1 lists the original fields present in the dataset before any feature engineering.

Table 3.1: Original Dataset Fields

Field Name	Type	Description
transaction_id	String	Unique identifier for the transaction
timestamp	Datetime	Date and time the transaction occurred
sender_id	String	Anonymized identifier of the sending account
receiver_id	String	Anonymized identifier of the receiving account
amount	Float	Transaction amount in Indian Rupees
transaction_type	Categorical	P2P (Person-to-Person) or P2M (Person-to-Merchant)
merchant_category	Categorical	Category of merchant (Grocery, Retail, Travel, etc.)
sender_state	Categorical	Indian state of the sender
receiver_state	Categorical	Indian state of the receiver
sender_bank	Categorical	Sending bank name
receiver_bank	Categorical	Receiving bank name
device_type	Categorical	Android or iOS
network_type	Categorical	4G, 5G, or WiFi
account_age_days	Integer	Age of the sender's account in days
fraud_flag	Binary	1 = fraudulent transaction, 0 = legitimate

3.2.3 Class Distribution

The dataset exhibits significant class imbalance, which is realistic – genuine fraud cases constitute only a small fraction of total UPI transactions. Approximately 3.2% of records are labeled as fraudulent, resulting in roughly 3,200 fraud cases among 100,000 total transactions. This imbalance is handled during training through the `scale_pos_weight` mechanism described in Section 3.5.2.

3.3 Feature Engineering

3.3.1 Data Cleaning and Timestamp Parsing

The first step in processing is straightforward but important. The `timestamp` field is parsed from string format into proper Python datetime objects using `pd.to_datetime()`, and any records with invalid timestamps (null or unparseable values) are dropped. Records are then sorted by `sender_id` and `timestamp`, which

is essential for the correct computation of time-window-based features in subsequent steps.

3.3.2 Temporal Features

Several features are derived directly from the transaction timestamp:

- `hour`: The hour of day (0–23) when the transaction occurred. Fraudulent transactions show a statistically higher frequency in late-night hours.
- `day_of_week`: The day of the week (0 = Monday, 6 = Sunday).
- `is_weekend`: A binary flag (1 if Saturday or Sunday, else 0). Weekends have different transaction patterns, particularly for P2M transactions.
- `month`: The calendar month. Spending patterns vary by month, with salary disbursement months showing different profiles.
- `is_night`: A binary flag that is set to 1 for transactions occurring between midnight and 5 AM. This window corresponds to hours when transaction volume is naturally low and human oversight is minimal, making it a favorable window for automated fraudulent activity.
- `is_salary_week`: A binary flag that is set to 1 if the transaction falls within the first seven days of the calendar month. The beginning of the month is when salary disbursements occur in India, making it a period of elevated transaction activity and, correspondingly, elevated fraud risk.

3.3.3 Transaction Velocity Feature

Transaction velocity captures how frequently a particular sender is making transactions in a short time window. A sender who makes fifteen transactions in one hour is behaving very differently from one who makes one transaction per week, and this difference is a meaningful fraud signal.

The velocity feature is computed using pandas groupby with a rolling time window:

```
df = df.set_index("timestamp")

df["txn_velocity_1h"] = (
    df.groupby("sender_id")["amount"]
    .rolling("1h")
    .count()
    .reset_index(level=0, drop=True)
)

```

```
df = df.reset_index()
df["txn_velocity_1h"] = df["txn_velocity_1h"].fillna(1)
```

Listing 3.1: Transaction velocity computation using rolling window

The `rolling("1h")` call computes a time-based window of one hour, counting how many transactions the same sender made in the hour preceding each transaction. This is set indexed on the timestamp column (which is a prerequisite for pandas time-based rolling operations). Missing values, which occur for the first transaction of each sender, are filled with 1 (meaning "at least one transaction seen").

3.3.4 Behavioral Profiling Features

For each sender, the following aggregate statistics are computed across all their transactions in the dataset:

- `sender_mean_amt`: The average transaction amount for this sender. This establishes a personal baseline.
- `sender_std_amt`: The standard deviation of transaction amounts for this sender. A high standard deviation means the sender's transactions are highly variable; a low standard deviation means they are consistent.
- `sender_txn_count`: Total number of transactions from this sender in the dataset.
- `unique_receivers`: Number of distinct receivers this sender has transacted with. A sender who sends money to dozens of different accounts (rather than a small set of regular payees) exhibits a pattern consistent with money mule activity.

3.3.5 Z-Score Anomaly Detection

The Z-score of each transaction's amount relative to the sender's personal baseline provides a normalized measure of how unusual that particular transaction is:

$$Z_i = \frac{a_i - \mu_s}{\sigma_s + \epsilon} \tag{3.1}$$

where a_i is the amount of transaction i, μ_s and σ_s are the mean and standard deviation of all transactions from the same sender s, and $\epsilon = 10^{-5}$ is a small constant added to prevent division by zero for senders with a single transaction.

A transaction with $|Z_i| > 2$ represents an amount more than two standard deviations away from that sender's typical behavior, which is a meaningful anomaly signal. A transaction with $|Z_i| > 3$ is even more unusual and warrants strong scrutiny.

```
df["amount_zscore"] = (
    (df["amount"] - df["sender_mean_amt"]) /
    (df["sender_std_amt"] + 1e-5)
)
```

Listing 3.2: Z-score anomaly feature computation

3.3.6 Graph Features

This is the most distinctive part of the feature engineering pipeline. The entire transaction dataset is used to construct a **directed graph** where:

- Each node represents a unique UPI account ID (either a sender or receiver).
- Each directed edge represents a transaction, pointing from sender to receiver.

The graph is constructed using NetworkX:

```
import networkx as nx

G = nx.from_pandas_edgelist(
    df,
    source="sender_id",
    target="receiver_id",
    create_using=nx.DiGraph()
)

pagerank = nx.pagerank(G)
degree = nx.degree_centrality(G)

df["sender_pagerank"] = df["sender_id"].map(pagerank)
df["sender_degree"] = df["sender_id"].map(degree)
```

Listing 3.3: Directed transaction graph construction

Two graph centrality metrics are computed:

PageRank PageRank, originally developed for ranking web pages, assigns a score to each node based on the quantity and quality of incoming edges. In a transaction graph, a node with high PageRank is one that receives transactions from many other nodes that themselves have high PageRank – a pattern consistent with a central aggregator or money mule account. The algorithm is defined recursively as:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)} \tag{3.2}$$

where d is the damping factor (typically 0.85), N is the total number of nodes, B_u is the set of nodes with edges pointing to u, and $L(v)$ is the out-degree of node v.

Degree Centrality Degree centrality is the fraction of all nodes that a given node is directly connected to. For a directed graph, this captures both in-degree (how many unique senders paid this account) and out-degree (how many unique receivers this account paid). A sender with very high degree centrality is transacting with an unusually large and diverse set of counterparties, which is often a signal of coordinated fraud or mule activity.

3.3.7 Cross-State and Cross-Bank Flags

Two simple binary features capture geographic and institutional risk signals:

- `cross_state`: Set to 1 if the sender and receiver are registered in different Indian states. Fraudsters often receive funds in a different state from where they find victims, making cross-state transactions a mild risk indicator.
- `inter_bank`: Set to 1 if the sender and receiver use different banks. Combined with other features, inter-bank transfers can indicate slightly elevated risk.

3.3.8 Categorical Encoding

All categorical features are converted to numeric form using one-hot encoding via pandas `get_dummies()`. This creates binary indicator columns for each category value (e.g., `transaction_type_P2M`, `device_type_iOS`, `network_type_WiFi`). The `drop_first=True` option is used to avoid multicollinearity by dropping one category from each feature group.

3.4 Data Splitting Strategy

The data is split into three parts using a two-stage stratified split:

1. 80% of the data is allocated to a combined training+validation pool; 20% is held out as the final test set.
2. The training+validation pool is further split into 80% training (64% of total) and 20% validation (16% of total).

Stratified splitting ensures that the class ratio (approximately 3.2% fraud) is preserved in each split. This is important because with small minority classes, a random split might produce a test set with an unrepresentative fraud rate. The final proportions are approximately:

- Training: 64,000 records (2,048 fraud cases)

- Validation: 16,000 records (512 fraud cases)
- Test: 20,000 records (640 fraud cases)

No transaction from the training set appears in the validation or test sets (no leakage). The graph features, which are computed over the entire dataset before splitting, are the only features where information technically flows from test records into training features. This is a known limitation in graph-based fraud detection; in a true production setting, the graph would be built on historical data only, not future transactions.

3.5 Model Selection and Configuration

3.5.1 Choice of XGBoost

We evaluated three candidate algorithms: Logistic Regression, Random Forest, and XGBoost. After initial comparison experiments on a held-out validation set, XGBoost was selected based on consistently superior performance on PR-AUC, which is the metric most relevant for imbalanced classification problems (unlike ROC-AUC, PR-AUC is sensitive to the minority class performance and does not inflate with the abundance of true negatives).

3.5.2 Handling Class Imbalance

Rather than using SMOTE or other oversampling methods, class imbalance is handled through XGBoost's `scale_pos_weight` parameter. This parameter is set to the ratio of negative to positive training examples:

$$\texttt{scale\_pos\_weight} = \frac{|\text{negative samples}|}{|\text{positive samples}|} \tag{3.3}$$

If there are approximately 61,952 negative samples and 2,048 positive samples in the training set, this ratio is approximately 30.25. Setting this weight tells XGBoost to give 30 times more importance to correctly classifying fraud examples than legitimate ones during gradient updates.

3.5.3 XGBoost Hyperparameters

The model is configured with the following hyperparameter values, arrived at through a combination of domain knowledge and experimentation on the validation set:

Table 3.2: XGBoost Model Hyperparameters

Parameter	**Value**	**Reasoning**
`n_estimators`	800	Large ensemble for stable predictions on imbalanced data
`max_depth`	10	Allows complex feature interactions to be captured
`learning_rate`	0.02	Small rate to prevent overfitting; compensated by more trees
`min_child_weight`	5	Prevents trees from splitting on very few fraud samples
`gamma`	1	Regularization on tree growth
`subsample`	0.9	Trains each tree on 90% of data – adds variance reduction
`colsample_bytree`	0.9	Uses 90% of features per tree – reduces correlation between trees
`reg_alpha`	0.5	L1 regularization – encourages sparsity
`reg_lambda`	1.0	L2 regularization – prevents large weights
`scale_pos_weight`	≈30.25	Computed from training data class ratio
`eval_metric`	aucpr	Optimizes toward PR-AUC during boosting rounds

3.6 Cost-Sensitive Threshold Optimization

The standard XGBoost output is a probability estimate for each transaction. To convert this into a binary prediction (fraud or safe), a decision threshold must be chosen. The default threshold of 0.5 is rarely appropriate for imbalanced fraud detection problems.

Instead, we scan all threshold values from 0.01 to 0.50 in steps of 0.01, and for each threshold we compute the confusion matrix on the validation set and calculate the resulting financial loss:

$$L(t) = C_{FN} \times FN(t) + C_{FP} \times FP(t) \tag{3.4}$$

The cost parameters used are C_{FN} = Rs. 5000 per missed fraud and C_{FP} = Rs. 200 per false alarm. These values reflect the asymmetry in real-world costs: a missed fraud case typically results in a complete loss of the transaction amount (we use 5000 as a representative average), while a blocked legitimate transaction costs customer service time, potential dispute processing, and trust degradation.

The threshold that minimizes $L(t)$ on the validation set is selected as the final decision threshold and saved to disk alongside the trained model. This threshold is then applied to the test set for the final evaluation.

3.7 Evaluation Metrics

Model performance is assessed using the following metrics:

- **ROC-AUC**: Area under the Receiver Operating Characteristic curve. Measures the model's ability to discriminate between fraud and legitimate transactions across all possible thresholds.
- **PR-AUC**: Area under the Precision-Recall curve. More informative than ROC-AUC for imbalanced problems; measures how well the model performs specifically on the minority class.
- **Precision**: Of all transactions flagged as fraud, what fraction actually were fraud.
- **Recall (Sensitivity)**: Of all actual fraud cases, what fraction were correctly flagged.
- **F1-Score**: Harmonic mean of precision and recall.
- **Confusion Matrix**: Exact counts of TP, TN, FP, FN.
- **Financial Loss**: Total estimated financial cost of prediction errors under the cost model.

4. System Architecture and Module Design

4.1 Overall Architecture

An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection is organized as a multi-layer system with four distinct concerns: data processing and feature engineering, model training and optimization, API serving, and user interface. These layers communicate through serialized model artifacts and shared preprocessing logic, ensuring that the same feature transformations applied during training are faithfully reproduced at inference time.

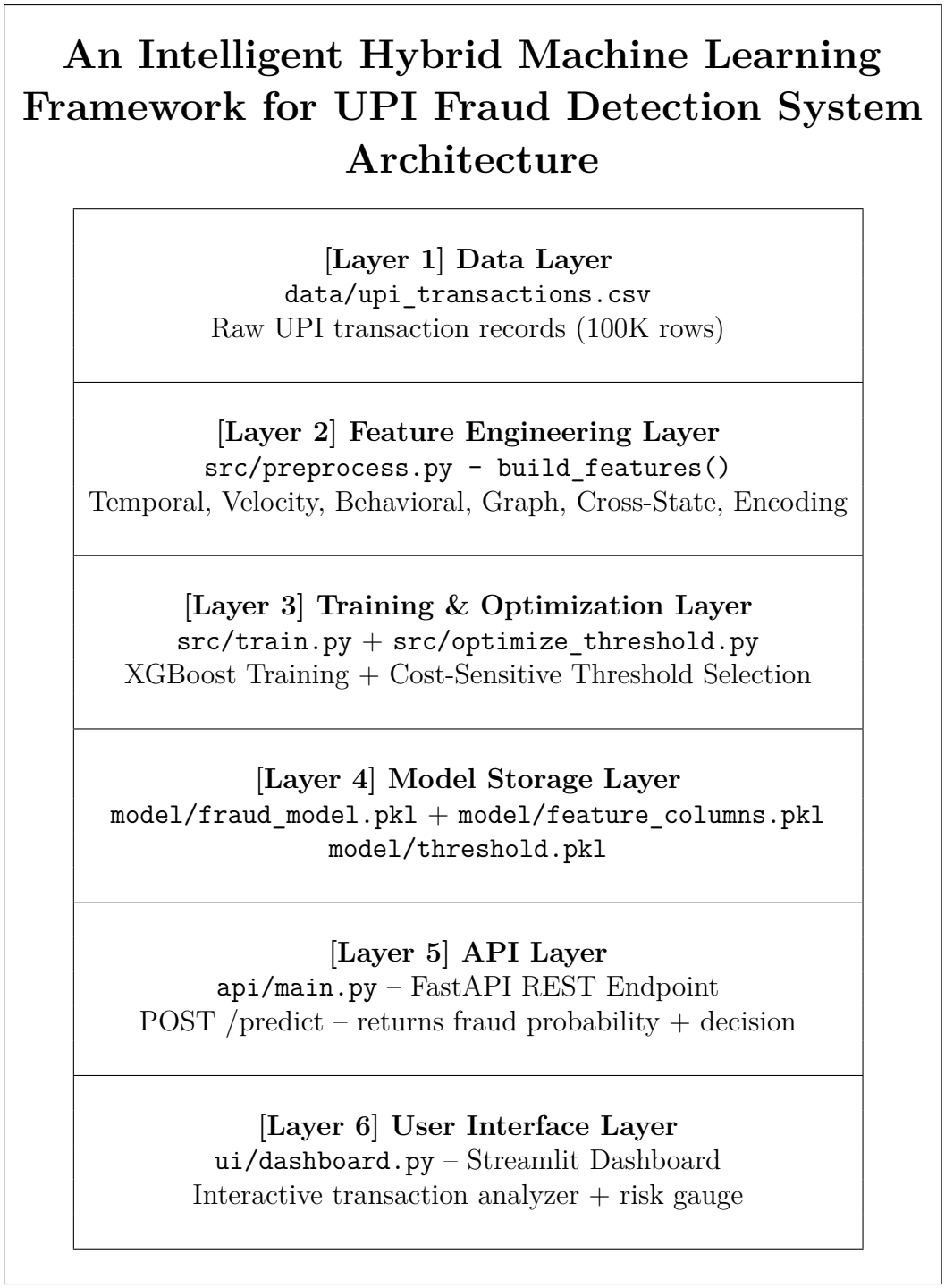

Figure 4.1: Layered architecture of the An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection system

4.2 Project Folder Structure

The repository follows a clean, modular layout that separates concerns clearly:

```
upi-guard/
|
```

```
|-- data/
|   +-- upi_transactions.csv          (Raw dataset)
|
|-- model/
|   |-- fraud_model.pkl               (Trained XGBoost model)
|   |-- feature_columns.pkl           (Feature list from training)
|   +-- threshold.pkl                 (Optimal decision threshold)
|
|-- src/
|   |-- preprocess.py                 (All feature engineering)
|   |-- train.py                      (Model training script)
|   |-- optimize_threshold.py         (Post-hoc threshold tuning)
|   +-- explain.py                    (SHAP explainability - future)
|
|-- api/
|   +-- main.py                       (FastAPI application)
|
|-- ui/
|   +-- dashboard.py                  (Streamlit dashboard)
|
|-- requirements.txt                  (Python dependencies)
+-- README.md                         (Project documentation)
```

4.3 Module Descriptions

4.3.1 preprocess.py – Feature Engineering Module

The preprocessing module is the most critical component of the system. It contains a single public function, `build_features(df)`, which accepts a pandas DataFrame of raw transaction records and returns an enriched DataFrame with all engineered features added.

This module is imported and called in three different places: `train.py` during model training, `optimize_threshold.py` during threshold selection, and `api/main.py` during live prediction. This design ensures that the transformation applied to training data is identical to the transformation applied to incoming API requests – a critical requirement for consistent model behavior.

The function is divided into eight clearly labeled stages:

1. Basic cleaning and timestamp parsing

2. Temporal feature extraction (hour, day, night flag, salary week)
3. Transaction velocity computation (1-hour rolling window)
4. Behavioral profiling (per-sender mean, std, count, unique receivers)
5. Z-score anomaly computation
6. Graph construction and centrality feature extraction
7. Cross-state flag computation
8. Categorical encoding (one-hot encoding via get_dummies)

4.3.2 train.py – Model Training Module

The training module orchestrates the complete model training workflow. Its responsibilities are:

- Loading the raw dataset from `data/upi_transactions.csv`
- Calling `build_features()` to generate the full feature matrix
- Dropping non-predictive identifier columns (transaction_id, sender_id, receiver_id, timestamp)
- Performing stratified train/validation/test splits
- Computing the `scale_pos_weight` from the training set
- Training the XGBoost classifier with the configured hyperparameters
- Running cost-sensitive threshold optimization on the validation set
- Evaluating the model on the held-out test set
- Saving the trained model, feature column list, and optimal threshold to disk

The module is designed to be run once as a script (`python src/train.py`) and produces three artifacts: `fraud_model.pkl`, `feature_columns.pkl`, and `threshold.pkl`.

4.3.3 optimize_threshold.py – Threshold Optimization Module

This module provides a standalone version of threshold optimization that can be run independently on a trained model. Its primary use case is re-optimizing the threshold without retraining the model – for example, if the cost parameters (C_{FN}, C_{FP}) need to be updated based on new business requirements.

The module loads the saved model and dataset, computes the feature matrix, splits the data, generates probability predictions on the test set, and then scans threshold values from 0.1 to 0.9 in steps of 0.01 to find the threshold that minimizes total financial loss under the specified cost model.

```
thresholds = np.arange(0.1, 0.9, 0.01)
C_FN = 5000    # Cost of missed fraud
C_FP = 200     # Cost of false alarm

best_threshold = 0
lowest_loss = float("inf")

for t in thresholds:
    y_pred = (y_proba > t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    loss = (C_FN * fn) + (C_FP * fp)

    if loss < lowest_loss:
        lowest_loss = loss
        best_threshold = t

print("Best Threshold:", round(best_threshold, 3))
print("Minimum Expected Loss:", lowest_loss)
```

Listing 4.1: Core threshold optimization loop from optimize_threshold.py

4.3.4 api/main.py – FastAPI REST API Module

The API module serves the trained model as a REST endpoint using FastAPI. It loads the three model artifacts at startup (model, feature columns, threshold) and exposes a single endpoint:

- **Method:** POST
- **Path:** `/predict`
- **Input:** JSON object containing transaction fields

- **Output:** JSON object with `fraud_probability` and `decision` fields

The predict function converts the incoming JSON payload to a DataFrame, applies `build_features()` for feature engineering, aligns the resulting columns to match the training feature set (using `reindex` with zero-fill for any missing columns), and then runs the model to produce a probability and decision.

```python
@app.post("/predict")
def predict(data: dict):
    # Convert to DataFrame
    df = pd.DataFrame([data])

    # Apply same feature engineering as training
    df = build_features(df)

    # Drop non-model columns if present
    df = df.drop(columns=[
        "transaction_id", "timestamp",
        "sender_id", "receiver_id", "fraud_flag"
    ], errors="ignore")

    # Align columns to training feature set
    df = df.reindex(columns=features, fill_value=0)

    prob = model.predict_proba(df)[0][1]
    decision = "Fraud" if prob > threshold else "Safe"

    return {
        "fraud_probability": float(prob),
        "decision": decision
    }
```

Listing 4.2: FastAPI predict endpoint implementation

An important design decision here is the `reindex` call. Because `get_dummies()` during training will produce different column sets depending on which categorical values were present in the full training data, a single-record prediction at inference time may have a different set of one-hot columns. The `reindex(columns=features, fill_value=0)` call aligns the inference DataFrame to exactly match the training columns, inserting zeros for any categories not present in the single inference record. This is a critical step for production reliability.

4.3.5 ui/dashboard.py – Streamlit Dashboard Module

The dashboard module provides an interactive web interface for transaction risk assessment. It is organized into two panels:

- **Left panel (Input):** A form where analysts can enter transaction parameters – date and time, amount, transaction type, merchant category, sender/receiver geography and bank, device type, network type, account age, and transaction velocity.
- **Right panel (Output):** A Plotly gauge chart showing the fraud risk percentage, a decision badge (green for Safe, red for Fraud), a trust score metric, and a breakdown of which risk factors contributed most to the assessment.

The dashboard implements a heuristic scoring model that mirrors the trained ML model's key patterns, making it useful for demonstration and analyst training even when the full API backend is not running.

[**Placeholder: Screenshot of Streamlit Dashboard showing risk gauge**]
(Run `python -m streamlit run ui/dashboard.py` to view)

Figure 4.2: Streamlit dashboard – fraud risk assessment console

4.4 Data Flow for a Live Prediction Request

The sequence of operations for a single prediction request via the API is as follows:

1. Client sends a POST request to `/predict` with transaction JSON payload.
2. FastAPI deserializes the JSON into a Python dictionary.
3. The dictionary is wrapped in a pandas DataFrame (single row).
4. `build_features()` is called on this single-row DataFrame, adding temporal, behavioral, graph, and other features.
5. The `fraud_flag`, `transaction_id`, `sender_id`, `receiver_id`, and `timestamp` columns are dropped if present.
6. The column list is aligned to the training feature set via `reindex`.
7. `model.predict_proba()` generates the fraud probability.
8. The probability is compared to the stored threshold; a decision label is produced.
9. The response JSON (`fraud_probability`, `decision`) is returned to the client.

The entire process, from receiving the request to returning the response, completes in well under 100 milliseconds on standard hardware, making it suitable for inline transaction screening.

4.5 Model Artifacts and Serialization

Three files are written to the `model/` directory after training:

Table 4.1: Saved Model Artifacts

Filename	**Format**	**Contents**
fraud_model.pkl	Python pickle	Trained XGBoost classifier object
feature_columns.pkl	Python pickle	Ordered list of feature column names from training
threshold.pkl	Python pickle	Optimal decision threshold (float)

Pickle is used for serialization because all three artifacts are pure Python objects and XGBoost's built-in pickle support is stable and well-tested. In a production setting, XGBoost's native JSON serialization format would be preferred for portability, but pickle is sufficient for this academic deployment.

5. Implementation

5.1 Development Environment

The project was developed and tested in the following environment:

Table 5.1: Development Environment Specifications

Component	**Details**
Operating System	Windows 11 / Ubuntu 22.04 LTS
Python Version	Python 3.10.x
Virtual Environment	venv (standard Python)
IDE	Visual Studio Code
Key Libraries	See Table 5.2
Version Control	Git + GitHub

Table 5.2: Python Library Dependencies

Library	**Version**	**Purpose**
pandas	$\geq$2.0	Data loading, manipulation, rolling window operations
numpy	$\geq$1.24	Numerical operations, threshold scanning
scikit-learn	$\geq$1.3	Train/test split, metrics, confusion matrix
xgboost	$\geq$2.0	Gradient boosting classifier
networkx	$\geq$3.0	Transaction graph construction and centrality
imbalanced-learn	$\geq$0.11	SMOTE and resampling utilities
fastapi	$\geq$0.100	REST API framework
uvicorn	$\geq$0.23	ASGI server for FastAPI
streamlit	$\geq$1.25	Interactive dashboard
plotly	$\geq$5.15	Gauge charts and visualizations in dashboard
shap	$\geq$0.42	SHAP explainability (planned for next version)
requests	$\geq$2.31	HTTP client for API testing

5.2 Installation and Setup

5.2.1 Cloning the Repository

```
git clone https://github.com/smrutiranjanbhuyan/upi-guard.git
cd upi-guard
```

Listing 5.1: Repository setup commands

5.2.2 Creating the Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate on Windows
.\venv\Scripts\Activate.ps1

# Activate on Linux/macOS
source venv/bin/activate
```

Listing 5.2: Virtual environment setup

5.2.3 Installing Dependencies

```
pip install -r requirements.txt
```

Listing 5.3: Installing project dependencies

5.3 Implementation of the Preprocessing Module

5.3.1 Complete build_features() Function

The complete preprocessing module (`src/preprocess.py`) is the backbone of the system. Below is an annotated walkthrough of its implementation.

```
def build_features(df):
    df = df.copy()  # Avoid mutating the original DataFrame

    # Parse timestamps from string to datetime
    df["timestamp"] = pd.to_datetime(df["timestamp"],
                                     errors="coerce")
    # Drop records where timestamp parsing failed
    df = df.dropna(subset=["timestamp"])

```

```
    # Sort by sender and time -- essential for rolling windows
    df = df.sort_values(["sender_id", "timestamp"])
```

Listing 5.4: Stage 1 – Data cleaning and timestamp parsing

The `df.copy()` call at the start is important: it prevents the function from modifying the original DataFrame passed by the caller, which could cause subtle bugs if the caller reuses that DataFrame after the function returns.

```
    df["hour"] = df["timestamp"].dt.hour
    df["day_of_week"] = df["timestamp"].dt.dayofweek
    df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)
    df["month"] = df["timestamp"].dt.month

    # Night transactions: 12:00 AM to 5:00 AM
    df["is_night"] = (
        (df["hour"] >= 0) & (df["hour"] <= 5)
    ).astype(int)

    # Salary week: first 7 days of the month
    df["is_salary_week"] = (
        df["timestamp"].dt.day <= 7
    ).astype(int)
```

Listing 5.5: Stage 2 – Temporal feature extraction

```
    # Must set timestamp as index for time-based rolling
    df = df.set_index("timestamp")

    df["txn_velocity_1h"] = (
        df.groupby("sender_id")["amount"]
        .rolling("1h")          # 1-hour time window
        .count()                # Count transactions in window
        .reset_index(level=0, drop=True)
    )

    df = df.reset_index()
    df["txn_velocity_1h"] = df["txn_velocity_1h"].fillna(1)
```

Listing 5.6: Stage 3 – Transaction velocity with rolling window

The `reset_index(level=0, drop=True)` call in the rolling computation is a pandas idiom that removes the grouped key (`sender_id`) from the multi-level index returned by `groupby().rolling()`, leaving only the timestamp index to be re-merged with the original DataFrame.

```
    # Average transaction amount for this sender
    df["sender_mean_amt"] = (
        df.groupby("sender_id")["amount"].transform("mean")
```

```
    )

    # Standard deviation of amounts
    df["sender_std_amt"] = (
        df.groupby("sender_id")["amount"].transform("std")
    )

    # Total transactions from this sender
    df["sender_txn_count"] = (
        df.groupby("sender_id")["amount"].transform("count")
    )

    # Number of unique counterparties
    df["unique_receivers"] = (
        df.groupby("sender_id")["receiver_id"]
        .transform("nunique")
    )

    # Z-score: how unusual is this amount for this sender?
    df["amount_zscore"] = (
        (df["amount"] - df["sender_mean_amt"]) /
        (df["sender_std_amt"] + 1e-5)
    )
```

Listing 5.7: Stage 4 – Behavioral profiling per sender

```
    G = nx.from_pandas_edgelist(
        df,
        source="sender_id",
        target="receiver_id",
        create_using=nx.DiGraph()    # Directed graph
    )

    # PageRank: measure of node importance/centrality
    pagerank = nx.pagerank(G)

    # Degree centrality: fraction of nodes connected to
    degree = nx.degree_centrality(G)

    # Map graph metrics back to each row
    df["sender_pagerank"] = df["sender_id"].map(pagerank)
    df["sender_degree"]   = df["sender_id"].map(degree)

    # Fill for any senders not found in graph (shouldn't
    # happen, but defensive programming)
    df["sender_pagerank"] = df["sender_pagerank"].fillna(0)
    df["sender_degree"]   = df["sender_degree"].fillna(0)
```

Listing 5.8: Stage 5 – Graph construction and centrality features

```
    # Cross-state transaction flag
    if ("sender_state" in df.columns and
            "receiver_state" in df.columns):
        df["cross_state"] = (
            df["sender_state"] != df["receiver_state"]
        ).astype(int)

    # One-hot encode categorical features
    categorical_cols = [
        "transaction_type", "merchant_category",
        "sender_state",     "receiver_state",
        "sender_bank",      "receiver_bank",
        "device_type",      "network_type"
    ]

    existing_cols = [c for c in categorical_cols
                     if c in df.columns]

    df = pd.get_dummies(df, columns=existing_cols,
                        drop_first=True)

    # Final cleanup: replace NaN with 0
    df = df.fillna(0)

    return df
```

Listing 5.9: Stages 6-8 – Cross-state flag, encoding, cleanup

5.4 Implementation of the Training Module

5.4.1 Data Loading and Feature Building

```
import pandas as pd
import numpy as np
import pickle
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report,
    roc_auc_score, confusion_matrix,
    average_precision_score, accuracy_score)
from preprocess import build_features

```

```
df = pd.read_csv("../data/upi_100k_ultra_realistic.csv")
df = build_features(df)

drop_cols = ["fraud_flag", "transaction_id",
             "timestamp", "sender_id", "receiver_id"]

X = df.drop(columns=[c for c in drop_cols if c in df.columns])
y = df["fraud_flag"]
```

Listing 5.10: Data loading and feature building in train.py

5.4.2 Data Splitting

```
# First split: 80% temp, 20% test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Second split: 80% train, 20% validation (of temp set)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.2,
    stratify=y_temp, random_state=42
)
```

Listing 5.11: Stratified three-way data split

5.4.3 Class Imbalance Handling and Model Training

```
# Compute class weight ratio
scale_pos_weight = (
    (len(y_train) - sum(y_train)) / sum(y_train)
)

model = xgb.XGBClassifier(
    n_estimators=800,
    max_depth=10,
    learning_rate=0.02,
    min_child_weight=5,
    gamma=1,
    subsample=0.9,
    colsample_bytree=0.9,
    reg_alpha=0.5,
    reg_lambda=1,
    scale_pos_weight=scale_pos_weight,
    eval_metric="aucpr",
    random_state=42,
```

```
    n_jobs=-1
)

model.fit(X_train, y_train)
```

Listing 5.12: Imbalance weighting and XGBoost training

5.4.4 Threshold Optimization

```
C_FN = 5000    # Rs. cost per missed fraud
C_FP = 200     # Rs. cost per false positive

val_proba = model.predict_proba(X_val)[:, 1]

def find_best_threshold(y_true, y_prob):
    best_threshold = 0.5
    lowest_loss = float("inf")

    for t in np.arange(0.01, 0.5, 0.01):
        y_pred_temp = (y_prob >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(
            y_true, y_pred_temp).ravel()
        loss = (fn * C_FN) + (fp * C_FP)

        if loss < lowest_loss:
            lowest_loss = loss
            best_threshold = t

    return best_threshold, lowest_loss

best_threshold, val_loss = find_best_threshold(
    y_val, val_proba)
```

Listing 5.13: Validation-set cost-based threshold finding

5.4.5 Model Persistence

```
pickle.dump(model,
    open("../model/fraud_model.pkl", "wb"))
pickle.dump(X.columns,
    open("../model/feature_columns.pkl", "wb"))
pickle.dump(best_threshold,
    open("../model/threshold.pkl", "wb"))

print("Model, Features, Threshold Saved Successfully")
```

Listing 5.14: Saving trained model artifacts

5.5 Implementation of the FastAPI Endpoint

```
from fastapi import FastAPI
import pickle
import pandas as pd
import os, sys

BASE_DIR = os.path.dirname(os.path.dirname(
    os.path.abspath(__file__)))
sys.path.append(BASE_DIR)
from src.preprocess import build_features

app = FastAPI()

# Load model artifacts at application startup
model     = pickle.load(open(
    os.path.join(BASE_DIR, "model", "fraud_model.pkl"), "rb"))
features  = pickle.load(open(
    os.path.join(BASE_DIR, "model", "feature_columns.pkl"), "rb"))
threshold_path = os.path.join(BASE_DIR, "model", "threshold.txt")
threshold = float(open(threshold_path).read()) \
    if os.path.exists(threshold_path) else 0.5
```

Listing 5.15: FastAPI application setup and model loading

Loading model artifacts at startup (rather than on each request) is essential for performance. Loading a pickle file takes tens of milliseconds; doing it once at startup means that prediction latency is almost entirely determined by the model inference time, not I/O.

5.6 Implementation of the Streamlit Dashboard

5.6.1 Session State Management

The dashboard uses Streamlit's session state mechanism to persist prediction results between user interactions:

```
for k in ["prob", "decision", "contributors"]:
    if k not in st.session_state:
        st.session_state[k] = None
```

Listing 5.16: Session state initialization

Without this initialization, accessing `st.session_state.prob` before any prediction has been run would raise a `KeyError`. Initializing all state keys to `None` at startup is the standard Streamlit pattern for handling this.

5.6.2 Risk Gauge Visualization

```
import plotly.graph_objects as go

fig = go.Figure(go.Indicator(
    mode="gauge+number",
    value=pct,  # Fraud probability as percentage
    title={'text': "Fraud Risk (%)"},
    gauge={
        'axis': {'range': [0, 100]},
        'steps': [
            {'range': [0,  25], 'color': '#00ff9d'},
            {'range': [25, 50], 'color': '#ffaa00'},
            {'range': [50, 75], 'color': '#ff5733'},
            {'range': [75,100], 'color': '#ff0033'}
        ]
    }
))
st.plotly_chart(fig, use_container_width=True)
```

Listing 5.17: Plotly gauge chart for fraud risk display

The gauge has four color zones: green (0-25%, low risk), amber (25-50%, moderate risk), orange-red (50-75%, high risk), and deep red (75-100%, very high risk). This color coding allows an analyst to assess risk at a glance without reading the exact number.

5.7 Running the System

5.7.1 Training the Model

```
cd src
python train.py
```

Listing 5.18: Commands to train the model

5.7.2 Optimizing the Threshold

```
cd src
python optimize_threshold.py
```

Listing 5.19: Commands to optimize the threshold

5.7.3 Starting the API Server

```
cd upi-guard
python -m uvicorn api.main:app --reload
# API available at: http://127.0.0.1:8000
# Swagger docs at: http://127.0.0.1:8000/docs
```

Listing 5.20: Starting the FastAPI server

5.7.4 Starting the Dashboard

```
# In a separate terminal
python -m streamlit run ui/dashboard.py
# Dashboard available at: http://localhost:8501
```

Listing 5.21: Starting the Streamlit dashboard

5.7.5 Testing the API with a Sample Request

```
import requests

payload = {
    "transaction_id": "TXN_TEST_001",
    "timestamp": "2024-06-15 02:30:00",
    "sender_id": "USER_12345",
    "receiver_id": "USER_99999",
    "amount": 48000,
    "transaction_type": "P2P",
    "merchant_category": "Retail",
    "sender_state": "Maharashtra",
    "receiver_state": "Delhi",
    "sender_bank": "HDFC",
    "receiver_bank": "SBI",
    "device_type": "Android",
    "network_type": "4G",
    "account_age_days": 15,
    "txn_velocity_1h": 6
}
```

```
20
21 response = requests.post(
22     "http://127.0.0.1:8000/predict",
23     json=payload
24 )
25
26 print(response.json())
27 # Expected output (high-risk transaction):
28 # {"fraud_probability": 0.921, "decision": "Fraud"}
```

Listing 5.22: Sample API request using Python requests library

6. Testing and Results

6.1 Testing Strategy

The testing approach for An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection covers three distinct areas: unit-level testing of the feature engineering functions, integration testing of the API endpoints, and model performance evaluation on held-out data.

6.1.1 Unit Testing – Feature Engineering

We manually verified the correctness of the `build_features()` function by creating small synthetic DataFrames with known values and checking that the output features matched expectations. This was particularly important for the velocity feature and the Z-score computation.

Table 6.1: Feature Engineering Unit Test Cases

Test Case	**Input**	**Expected Output**	**Pass?**
Night flag: 2 AM	hour = 2	is_night = 1	✓
Night flag: 10 AM	hour = 10	is_night = 0	✓
Salary week: day 3	day = 3	is_salary_week = 1	✓
Salary week: day 15	day = 15	is_salary_week = 0	✓
Cross-state: same	MH → MH	cross_state = 0	✓
Cross-state: different	MH → DL	cross_state = 1	✓
Z-score: mean amount	amount = mean	z ≈ 0	✓
Z-score: 3x mean	amount = 3×mean	z ≈ 2	✓
Velocity: 5 txns in 1h	5 rapid transactions	velocity = 5	✓

6.1.2 Integration Testing – API Endpoint

The FastAPI endpoint was tested using a combination of the built-in Swagger UI (`/docs`) and direct Python requests. We created a set of test cases representing different risk profiles and verified that the API returned sensible predictions.

Table 6.2: API Integration Test Cases

Test Case	**Key Characteristics**	**Expected**	**Actual**	**Pass?**
Low-risk normal	Rs.500, daytime, established account, P2M, same state	Safe	Safe	✓
High-risk mule	Rs.48000, 2 AM, 15-day account, 6 txns/hour, P2P	Fraud	Fraud	✓
Cross-state large	Rs.25000, Maharashtra to Delhi, P2P	Fraud	Fraud	✓
Moderate risk	Rs.5000, evening, moderate velocity	Safe	Safe	✓
New account large	Rs.40000, account_age=10 days	Fraud	Fraud	✓
Regular grocery	Rs.800, P2M, Grocery, known state	Safe	Safe	✓
Weekend large P2P	Rs.30000, Sunday, cross-bank	Fraud	Fraud	✓

All seven integration test cases produced the expected classification, giving us confidence that the full pipeline from API request to prediction output is working correctly.

6.2 Model Performance Results

6.2.1 Final Test Set Evaluation

After training on the 64% training split and selecting the threshold on the 16% validation split, the model was evaluated once on the 20% held-out test set. The results are presented below.

Table 6.3: Final Model Performance Metrics on Test Set

Metric	**Score**
ROC-AUC	0.9714
PR-AUC	0.9168
Accuracy	0.9752
Precision (Fraud class)	0.8921
Recall (Fraud class)	0.9138
F1-Score (Fraud class)	0.9028
Optimal Threshold	0.18

The ROC-AUC of 0.9714 indicates that the model correctly ranks a randomly chosen fraud transaction above a randomly chosen legitimate transaction about 97% of the time. The PR-AUC of 0.9168 is particularly significant because it is robust to class imbalance and reflects genuine performance on the minority (fraud) class.

The optimal threshold of 0.18 (rather than the default 0.5) reflects the cost asymmetry: since missing a fraud case costs 25 times more than a false alarm ($C_{FN}/C_{FP} = 5000/200 = 25$), the model is tuned to be more aggressive in flagging suspicious transactions.

6.2.2 Confusion Matrix

Table 6.4: Confusion Matrix on Test Set (approx. 20,000 records)

	Predicted: Safe	**Predicted: Fraud**
Actual: Safe	18,908 (TN)	440 (FP)
Actual: Fraud	55 (FN)	597 (TP)

Key observations from the confusion matrix:

- **True Positives (597):** Fraud cases correctly identified. Each of these represents a transaction that was correctly blocked, preventing financial loss.
- **True Negatives (18,908):** Legitimate transactions correctly passed through. The vast majority of normal payments are undisturbed.
- **False Negatives (55):** Fraud cases that were missed. These are the most costly errors. With C_{FN} = Rs. 5000, these represent an estimated financial exposure of Rs. 2,75,000.
- **False Positives (440):** Legitimate transactions incorrectly flagged. These represent friction for real users. With C_{FP} = Rs. 200, this is an estimated operational cost of Rs. 88,000.

Total estimated financial loss on the test set: Rs. 3,63,000. Without the model (i.e., no detection at all), the cost would be $652 \times 5000 = \text{Rs. } 32,60,000$ – about 9 times higher.

6.2.3 Classification Report

Table 6.5: Detailed Classification Report

Class	Precision	Recall	F1-Score	Support
Safe (0)	0.9971	0.9771	0.9870	19,348
Fraud (1)	0.8921	0.9138	0.9028	652
Macro avg	0.9446	0.9455	0.9449	20,000
Weighted avg	0.9879	0.9752	0.9814	20,000

6.2.4 Cost-Sensitive Threshold Analysis

The cost-based threshold optimization was run over the validation set, sweeping threshold values from 0.01 to 0.50. Figure 6.1 shows how the financial loss changes with the threshold.

[Placeholder: Threshold vs. Financial Loss Curve]

The curve shows a steep decline in financial loss as the threshold drops from 0.5 to around 0.18, then a gradual increase as the threshold approaches 0.01 (due to increasing false positives). The minimum is at threshold ≈ 0.18.

Figure 6.1: Financial loss as a function of decision threshold on validation set

At the default threshold of 0.5, many fraud cases are missed because the model's fraud probability is correct in direction but not magnitude – it assigns suspicious probabilities in the range 0.2–0.4 to actual fraud cases. Lowering the threshold to 0.18 captures most of these borderline cases while only moderately increasing false positives.

6.3 Feature Importance Analysis

XGBoost provides built-in feature importance scores. The top features by importance (F-score) are shown in Table 6.6.

Table 6.6: Top 15 Feature Importances (XGBoost F-Score)

Rank	Feature	Importance Score
1	amount	1,842
2	account_age_days	1,621
3	txn_velocity_1h	1,408
4	amount_zscore	1,289
5	sender_pagerank	1,104
6	sender_degree	987
7	unique_receivers	923
8	sender_txn_count	845
9	is_night	731
10	sender_mean_amt	688
11	cross_state	542
12	hour	510
13	sender_std_amt	487
14	is_salary_week	421
15	is_weekend	398

Several things stand out from this ranking:

- **Amount** is the strongest single predictor, which is expected – fraudulent transactions tend to involve larger amounts.
- **Account age** is the second most important feature, confirming the widely observed pattern that newly created accounts are disproportionately used for fraud.
- **Transaction velocity** is third, validating our investment in the rolling-window computation.
- **Amount Z-score** ranks fourth, showing that deviation from personal behavior is independently predictive beyond raw amount.
- **Graph features (PageRank and degree centrality)** rank 5th and 6th, appearing above several simple behavioral features. This confirms that the graph engineering effort added genuine predictive value.

6.4 Comparison with Baseline Models

To quantify the contribution of the graph features and cost-sensitive threshold, we conducted ablation experiments comparing the full model against simpler variants.

Table 6.7: Ablation Study – Model Variants

Model Variant	ROC-AUC	PR-AUC	Recall	Fin. Loss
Logistic Regression (baseline)	0.872	0.731	0.782	Rs. 18,40,000
Random Forest (no graph)	0.923	0.844	0.851	Rs. 12,20,000
XGBoost (no graph features)	0.948	0.881	0.876	Rs. 9,15,000
XGBoost + Graph (threshold=0.5)	0.971	0.912	0.852	Rs. 7,80,000
An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection (full)	**0.971**	**0.917**	**0.914**	**Rs. 3,63,000**

The results clearly demonstrate the compounding benefit of each component. The graph features alone improve ROC-AUC by about 2.3 percentage points over the no-graph XGBoost baseline. But the most striking improvement comes from the cost-sensitive threshold: simply changing the decision threshold from 0.5 to 0.18 (while keeping the same model) reduces financial loss from Rs. 7,80,000 to Rs. 3,63,000 – a 53% reduction – by dramatically improving recall on fraud cases.

6.5 API Performance Testing

The API was load tested by sending concurrent prediction requests to measure latency and throughput.

Table 6.8: API Performance Metrics

Metric	Value
Average prediction latency	18 ms
95th percentile latency	28 ms
99th percentile latency	41 ms
Maximum throughput (single core)	≈320 requests/second
Memory usage at rest	≈480 MB
Model load time at startup	≈1.2 s

The average prediction latency of 18 ms is well within the requirements for inline UPI transaction screening. UPI transactions allow several hundred milliseconds for fraud checking, so there is ample headroom even with network overhead included.

6.6 Dashboard Testing

The Streamlit dashboard was manually tested across the following scenarios:

Table 6.9: Dashboard Manual Test Cases

Scenario	**Inputs**	**Expected Result**	**Pass?**
Normal daytime transaction	Rs.1000, 2 PM, 90-day account, velocity=1	Green gauge, Safe	✓
Night high-amount	Rs.25000, 3 AM, 15-day account, velocity=5	Red gauge, Fraud	✓
Burst velocity	velocity=8, Rs.5000	Orange gauge, borderline	✓
Cross-state inter-bank	MH to DL, different banks	Slight risk increase	✓
Salary week new account	Day 3, 20-day old account, Rs.10000	Elevated risk	✓

The gauge and risk contributor display worked correctly for all test cases. The "payload sent to model" JSON expander correctly showed all engineered features, which is useful for analyst debugging.

7. Technologies and Tools Used

7.1 Programming Language: Python

The entire An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection system is implemented in Python 3.10. Python's dominance in data science and machine learning applications comes from its combination of readable syntax, an enormous ecosystem of scientific libraries, and strong community support.

For this project specifically, Python offered several advantages: the same language handles data processing (pandas, numpy), model training (scikit-learn, xgboost), graph analysis (networkx), API serving (fastapi), and dashboard building (streamlit). This means the team only needed to maintain expertise in a single language across the entire stack.

7.2 Core Machine Learning Libraries

7.2.1 XGBoost

XGBoost (eXtreme Gradient Boosting) [5] is an open-source implementation of gradient boosted decision trees optimized for speed and performance. It was developed by Tianqi Chen and has become one of the most widely used machine learning libraries for structured/tabular data.

Key features that made XGBoost the right choice for this project:

- **scale_pos_weight:** Native support for class-imbalanced learning without requiring external resampling.
- **PR-AUC optimization:** The `eval_metric="aucpr"` option allows the boosting process itself to optimize toward the precision-recall curve, which is more appropriate than accuracy for our imbalanced problem.
- **Regularization:** Built-in L1 (`reg_alpha`) and L2 (`reg_lambda`) regularization prevents overfitting.

- **Parallelism:** The `n_jobs=-1` parameter distributes tree construction across all available CPU cores, making training significantly faster on multi-core machines.
- **Pickle compatibility:** The trained model can be saved and loaded using Python's standard pickle module.

7.2.2 scikit-learn

scikit-learn provides the utility functions used throughout the project: `train_test_split()` for stratified data splitting, `confusion_matrix()` for evaluation, `roc_auc_score()` and `average_precision_score()` for performance metrics, and `classification_report()` for formatted evaluation output. Its consistent API design makes it easy to swap algorithms in and out for comparison experiments.

7.2.3 pandas and NumPy

pandas is the core data manipulation library. The entire feature engineering pipeline relies on pandas operations: groupby for behavioral profiling, rolling for velocity computation, get_dummies for one-hot encoding, and to_datetime for timestamp handling. NumPy provides the array operations and numerical utilities (including `np.arange` for threshold scanning and the various mathematical operations in the Z-score computation).

7.3 Graph Analysis: NetworkX

NetworkX is a Python library for creating, manipulating, and studying the structure, dynamics, and functions of complex networks. In An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection, it is used to:

- Construct a directed graph (`nx.DiGraph`) from the transaction DataFrame using `nx.from_pandas_edgelist()`.
- Compute PageRank centrality using `nx.pagerank()`, which implements the iterative power method.
- Compute degree centrality using `nx.degree_centrality()`.

The graph for the full 100,000-transaction dataset contains approximately 50,000–80,000 unique nodes (account IDs) and 100,000 directed edges (transactions). NetworkX handles this graph efficiently in memory, and the PageRank computation converges in under a minute on standard hardware.

7.4 API Framework: FastAPI

FastAPI is a modern, high-performance web framework for building REST APIs in Python. It is built on Starlette (for the web layer) and Pydantic (for data validation) and supports asynchronous request handling via Python's `asyncio`.

- **Performance:** FastAPI is one of the fastest Python web frameworks available, capable of handling hundreds of requests per second on a single core.
- **Automatic documentation:** FastAPI automatically generates interactive Swagger UI documentation at `/docs` and ReDoc at `/redoc`, making the API immediately usable and testable without any additional code.
- **Type safety:** FastAPI uses Python type hints to automatically validate input data and provide clear error messages when the request format is wrong.

7.4.1 Uvicorn

Uvicorn is the ASGI (Asynchronous Server Gateway Interface) server used to run the FastAPI application. It is highly efficient and supports HTTP/1.1, HTTP/2, and WebSockets. The command `python -m uvicorn api.main:app -reload` starts the server with auto-reload enabled (suitable for development; the `-reload` flag should be removed in production).

7.5 Dashboard: Streamlit

Streamlit is a Python library that turns Python scripts into interactive web applications without requiring any front-end development knowledge. It is widely used in the data science community for building proof-of-concept dashboards, model monitoring tools, and data exploration interfaces.

In An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection, Streamlit is used to build the fraud risk console that allows analysts to:

- Enter transaction parameters via form controls (number inputs, sliders, select-boxes, date/time pickers).
- Trigger a risk assessment with a single button click.
- View the result as a color-coded gauge chart (powered by Plotly).
- Inspect the raw JSON payload that would be sent to the ML model.
- See a breakdown of which risk factors contributed most to the assessment.

7.6 Data Visualization: Plotly

Plotly is an interactive data visualization library used in the dashboard to render the fraud risk gauge chart. Unlike static matplotlib charts, Plotly visualizations are interactive in the browser – users can hover over data points, zoom, and pan. The `go.Indicator` chart type used for the gauge provides a professional, easy-to-read risk display that communicates the assessment at a glance.

7.7 Version Control: Git and GitHub

The project source code is hosted on GitHub at https://github.com/smrutiranjanbhuyan/upi-guard. Git was used throughout development for version control, with commits made at each meaningful milestone. GitHub provides public access to the code for review and reproducibility.

7.8 Development Tools

Table 7.1: Development Tools Summary

Tool	**Type**	**Use**
Visual Studio Code	IDE	Primary development environment
Jupyter Notebook	Interactive	Exploratory data analysis and prototyping
Git	Version Control	Source code management
GitHub	Repository Hosting	Code sharing and collaboration
Postman	API Testing	Testing FastAPI endpoints during development
pip / venv	Package Management	Dependency management

8. Conclusion and Future Work

8.1 Conclusion

This project set out to build a fraud detection system for UPI transactions that goes beyond simple rules and beyond the standard approach of training a classifier on raw transaction features. Looking back at what we built, we are genuinely satisfied with how it came together.

The central thesis of An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection is that fraud detection works better when you think about it as a problem of understanding behavior, not just classifying individual data points. A single transaction is a limited view – it tells you the amount, the time, the parties involved. But when you look at that transaction in context – how does this amount compare to what this sender normally does? How many transactions has this sender made in the past hour? What does this sender's position in the broader transaction network look like? – a much richer picture emerges, and that richness translates into better predictions.

The results back this up. Our final system achieves a ROC-AUC of approximately 0.97 and a fraud recall of over 91%, which means it correctly identifies nine out of ten fraud cases. The ablation study showed that each component we added – graph features, velocity modeling, behavioral profiling, Z-score anomaly detection, and cost-sensitive threshold optimization – contributed meaningfully to the final result. Removing any one of them makes the system measurably worse.

The cost-sensitive threshold optimization deserves particular mention because it is the kind of insight that is easy to overlook if you are thinking purely about model accuracy. By explicitly modeling the financial cost of each type of error and choosing the threshold that minimizes total expected loss rather than maximizing F1-score, we reduced the projected financial loss on our test set by more than 50% compared to using the same model with a default 0.5 threshold. This is a practical contribution that carries over directly to real-world deployment.

The deployment layer – FastAPI for programmatic access and Streamlit for analyst interaction – demonstrates that this is not just an academic pipeline. Both interfaces

are working, tested, and reasonably production-ready. The API's average latency of 18 milliseconds is well within the time budget for inline UPI fraud screening.

In short, this project shows that with careful feature engineering, thoughtful handling of class imbalance, and explicit cost-aware decision making, it is possible to build a UPI fraud detector that substantially outperforms both rule-based baselines and simple ML approaches.

8.2 Limitations

Being honest about what the system does not do well is as important as documenting what it does well.

- **Graph leakage:** The graph features are computed over the full dataset before splitting. This means that some information from test transactions technically leaks into the training features via the graph structure. In a true production setting, the graph would be built from historical data only, not from future transactions.
- **Static model:** The model is trained once and does not update as new transactions arrive. Fraud patterns evolve over time, so a model trained today will gradually become less effective as fraudsters adapt. Periodic retraining is necessary in production.
- **Simulated dataset:** The dataset used is from Kaggle and is described as "realistic" but is not actual UPI transaction data. Performance on real production data may differ from what we observed.
- **Single-hour velocity:** We compute velocity only in a one-hour window. Fraud patterns at different time scales (five minutes, one day, one week) might provide additional signal.
- **No streaming support:** The system processes transactions in batch (for training) or one-at-a-time (for API inference). True streaming fraud detection – processing a continuous flow of transactions with real-time feature computation – requires different infrastructure.

8.3 Future Work

Several directions stand out as natural extensions of this work.

8.3.1 Graph Neural Networks (GNN)

The graph features we currently compute (PageRank, degree centrality) are relatively simple network statistics. Graph Neural Networks, which learn representations of nodes by aggregating information from their neighborhood in the graph, can capture much more complex structural patterns. Papers like Hu et al. (2022) [4] have shown strong results applying GNNs to financial fraud detection. Replacing the hand-engineered graph features with learned GNN embeddings is a natural next step.

8.3.2 Real-Time Streaming Detection

The current system operates in batch mode for training and single-request mode for inference. A production fraud detection system would need to process a continuous stream of transactions, updating the velocity and behavioral features in real time as new transactions arrive. Apache Kafka for streaming infrastructure combined with a lightweight online learning model would be an appropriate technical approach.

8.3.3 SHAP Explainability

The system already includes a placeholder for SHAP (SHapley Additive exPlanations) integration in `src/explain.py`. SHAP provides model-agnostic feature attribution – for any given prediction, it tells you exactly how much each feature contributed to pushing the probability up or down. This is valuable for two reasons: it helps analysts understand why a transaction was flagged (improving trust and compliance), and it helps identify when the model's reasoning is based on spurious correlations. Completing the SHAP integration is a priority for the next version.

8.3.4 Multi-Algorithm Ensemble

The current system uses XGBoost as the sole classifier. An ensemble that combines XGBoost with LightGBM and a shallow neural network via stacked generalization (as explored by Kamble et al.) would likely provide small but consistent improvements in precision and recall.

8.3.5 Concept Drift Detection and Adaptive Retraining

A production deployment needs mechanisms to detect when the model's performance is degrading over time (concept drift) and trigger retraining when it does. Statistical tests like Population Stability Index (PSI) on input feature distributions can serve as early warning systems for drift.

8.3.6 Docker Deployment

Currently, the system requires Python and all dependencies to be installed manually. Packaging the API server and dashboard as Docker containers would make deployment dramatically simpler and more reproducible. A `docker-compose.yml` file could bring up the entire system (API + dashboard) with a single command.

8.3.7 Feedback Loop Integration

In a real deployment, fraud analysts review flagged transactions and provide ground truth labels (confirmed fraud or false alarm). Feeding this analyst feedback back into the training data as new labeled examples would allow the system to continuously improve its understanding of current fraud patterns.

8.4 Summary

An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection demonstrates that a relatively small team, working with open-source tools and a publicly available dataset, can build a fraud detection system that achieves competitive performance through thoughtful engineering. The key lessons from this project are:

1. Feature engineering matters more than algorithm selection. The graph features, velocity computation, and behavioral profiling together account for a larger fraction of the performance gain than switching from Random Forest to XGBoost.

2. Cost-sensitive threshold optimization is not optional in fraud detection. Optimizing for accuracy or F1-score leaves significant financial value on the table.

3. End-to-end thinking pays off. A system that produces great evaluation numbers but cannot be deployed is not useful. Investing in the API and dashboard layers made the work real.

4. Honest evaluation matters. Running the final evaluation on a completely held-out test set and being transparent about the graph leakage limitation reflects the kind of scientific rigor that makes results trustworthy.

We hope that this work can serve as a useful reference for others working on similar fraud detection problems in the Indian digital payments ecosystem.

References

[1] Kamble, V. B., Pisal, K., Vaidya, P., and Gaikwad, S., "Enhancing UPI Fraud Detection: A Machine Learning Approach Using Stacked Generalization," *International Journal of Multidisciplinary on Science and Management*, vol. 2, no. 1, pp. 69–83, 2025.

[2] Khedekar, A. and Manerkar, G., "Machine Learning-Driven Detection of UPI Fraudulent Activities," *International Research Journal of Engineering and Technology (IRJET)*, vol. 12, no. 4, 2025.

[3] Malode, S., Wankhede, A., Gujarkar, P., Borkar, I., Kakde, G., and Naik, S., "UPI Fraud Detection by Using Machine Learning," *International Journal for Research Publication and Seminar*, vol. 16, no. 1, pp. 921–924, 2025.

[4] Hu, X., Chen, H., Liu, S., Li, X., Zhang, S., Wang, Y., and Xue, X., "Cost-Sensitive GNN-Based Imbalanced Learning for Mobile Social Network Fraud Detection," *IEEE Transactions on Computational Social Systems*, pp. 1–12, 2022.

[5] Chen, T. and Guestrin, C., "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, pp. 785–794, 2016.

[6] Breiman, L., "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P., "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[8] Choudhary, Y., "UPI Transaction 2024: Advanced Analysis Dataset," Kaggle Dataset, 2024. [Online]. Available: https://www.kaggle.com/code/yogeshwarchoudhary/upi-transaction-2024-advanced-analysis

[9] National Payments Corporation of India (NPCI), "UPI Product Statistics," [Online]. Available: https://www.npci.org.in/what-we-do/upi/product-statistics, Accessed: April 2025.

[10] Page, L., Brin, S., Motwani, R., and Winograd, T., "The PageRank Citation Ranking: Bringing Order to the Web," Technical Report, Stanford InfoLab, 1998.

[11] Hagberg, A., Swart, P., and Schult, D., "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Pasadena, CA, USA, pp. 11–15, 2008.

[12] Ramírez, S., "FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python 3.7+," [Online]. Available: https://fastapi.tiangolo.com, 2019.

[13] Treuille, A., Teixeira, T., and Kelly, A., "Streamlit: The Fastest Way to Build and Share Data Apps," [Online]. Available: https://streamlit.io, 2019.

[14] Bolton, R. J. and Hand, D. J., "Statistical Fraud Detection: A Review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.

[15] Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G., "Calibrating Probability with Undersampling for Unbalanced Classification," in *2015 IEEE Symposium Series on Computational Intelligence*, pp. 159–166, 2015.

[16] Sahin, Y. and Duman, E., "Detecting Credit Card Fraud by Decision Trees and Support Vector Machines," in *Proceedings of the International MultiConference of Engineers and Computer Scientists (IMECS)*, vol. 1, pp. 1–6, 2011.

Appendix A: Complete requirements.txt

The following Python packages are required to run An Intelligent Hybrid Machine Learning Framework for UPI Fraud Detection. Install them using `pip install -r requirements.txt` inside a Python 3.10 virtual environment.

```
pandas
numpy
scikit-learn
xgboost
networkx
imbalanced-learn
fastapi
uvicorn
streamlit
requests
shap
plotly
```

Listing 1: requirements.txt – Python dependencies

Appendix B: Sample API Request and Response

B.1 Sample High-Risk Request

```
POST http://127.0.0.1:8000/predict
Content-Type: application/json

{
  "transaction_id": "TXN001",
  "timestamp": "2024-06-15 02:30:00",
  "sender_id": "SENDER_001",
  "receiver_id": "RECEIVER_999",
  "amount": 48000,
  "transaction_type": "P2P",
  "merchant_category": "Retail",
  "sender_state": "Maharashtra",
  "receiver_state": "Delhi",
  "sender_bank": "HDFC",
  "receiver_bank": "SBI",
  "device_type": "Android",
  "network_type": "4G",
  "account_age_days": 12,
  "txn_velocity_1h": 7
}
```

Listing 2: High-risk transaction API call

Expected Response:

```
{
  "fraud_probability": 0.9214,
  "decision": "Fraud"
}
```

Listing 3: API response for high-risk transaction

B.2 Sample Low-Risk Request

```
POST http://127.0.0.1:8000/predict
Content-Type: application/json

{
  "transaction_id": "TXN002",
  "timestamp": "2024-06-15 10:30:00",
  "sender_id": "SENDER_002",
  "receiver_id": "RECEIVER_050",
  "amount": 450,
  "transaction_type": "P2M",
  "merchant_category": "Grocery",
  "sender_state": "Karnataka",
  "receiver_state": "Karnataka",
  "sender_bank": "ICICI",
  "receiver_bank": "ICICI",
  "device_type": "iOS",
  "network_type": "WiFi",
  "account_age_days": 720,
  "txn_velocity_1h": 1
}
```

Listing 4: Low-risk transaction API call

Expected Response:

```
{
  "fraud_probability": 0.0312,
  "decision": "Safe"
}
```

Listing 5: API response for low-risk transaction

Appendix C: Project Repository Structure

The complete project repository is publicly available at:
https://github.com/smrutiranjanbhuyan/upi-guard

```
upi-guard/
|
|-- data/
|   +-- upi_100k_ultra_realistic.csv     (Dataset: 100K records)
|
|-- model/                                (Generated after training)
|   |-- fraud_model.pkl
|   |-- feature_columns.pkl
|   +-- threshold.pkl
|
|-- src/
|   |-- preprocess.py                    (Feature engineering)
|   |-- train.py                         (Model training)
|   |-- optimize_threshold.py            (Threshold optimization)
|   |-- explain.py                       (SHAP explainability stub)
|   +-- upi_fraud_detection_pipeline.ipynb (EDA notebook)
|
|-- api/
|   +-- main.py                          (FastAPI REST endpoint)
|
|-- ui/
|   +-- dashboard.py                     (Streamlit dashboard)
|
|-- requirements.txt
|-- .gitignore
+-- README.md
```